Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 4

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Трекинг множественных объектов»

Выполнил:
Студент 4 курса
Группы ИИ-26
Згера Е. А.
Проверил:
Андренко К.В.

Брест 2026

Вариант 5

Цель: исследовать применение алгоритмов трекинга на базе обученной сети-детектора объектов.

Задачи лабораторной работы

1. Используя сеть-детектор, обученный в ЛР 3, реализовать логику для отслеживания множественных объектов, используя библиотеку Ultralytics YOLO;
2. Применять алгоритмы BoT-Sort и ByteTrack (задействовать соответствующие конфигурационные файлы);
3. Исследовать изменения параметров в конфигурационных файлах и их влияние на качество трекинга;
4. В качестве исходных видеоматериалов для экспериментов использовать видео-ролики из сети (например, из YouTube), содержащие множественные объекты классов из ЛР 3;
5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

5	YOLO11m	**Счетчики расхода воды:** https://universe.roboflow.com/koer3741-gmail-com/watermeteramrv2/dataset/1

Код программы

```
import subprocess, sys
subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "ultralytics", "lapx", "roboflow",
"pyyaml"])

import os, time, shutil, collections, getpass
from pathlib import Path
import cv2
import yaml
import torch
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import ultralytics
from ultralytics import YOLO
from google.colab import files

DEVICE = "0" if torch.cuda.is_available() else "cpu"
IMGSZ = 640
DET_CONF = 0.10
DET_IOU = 0.5
MAX_FRAMES = 600
OUT_DIR = Path("lab4_out"); OUT_DIR.mkdir(exist_ok=True)
VIDEO_DIR = Path("videos"); VIDEO_DIR.mkdir(exist_ok=True)
CFG_DIR = Path("tracker_cfgs"); CFG_DIR.mkdir(exist_ok=True)

cands = [
    "runs/yolo11m_watermeter/weights/best.pt",
    "runs/detect/runs/yolo11m_watermeter/weights/best.pt",
    "/content/runs/detect/runs/yolo11m_watermeter/weights/best.pt",
    "/content/drive/MyDrive/lab_weights/best.pt",
    "best.pt",
]
WEIGHTS = next((p for p in cands if Path(p).exists()), None)

if WEIGHTS is None:
    from roboflow import Roboflow
    API_KEY = getpass.getpass("Roboflow API key: ").strip()
    EPOCHS, BATCH, RUN_NAME = 20, 16, "yolo11m_watermeter"
    rf = Roboflow(api_key=API_KEY)
    ds = rf.workspace("koer3741-gmail-com").project("watermeteramrv2").version(1).download(
```

```
        "yolov8", location="datasets/watermeteramrv2")
    ds_dir = Path(ds.location).resolve()
    dcfg = yaml.safe_load((ds_dir / "data.yaml").read_text(encoding="utf-8"))
    dcfg["path"] = str(ds_dir)
    for split in ("train", "val", "test"):
        if (ds_dir / split / "images").exists():
            dcfg[split] = f"{split}/images"
        else:
            dcfg.pop(split, None)
    if "val" not in dcfg and "test" in dcfg:
        dcfg["val"] = dcfg["test"]
    data_yaml = ds_dir / "data_fixed.yaml"
    data_yaml.write_text(yaml.safe_dump(dcfg, allow_unicode=True, sort_keys=False), encoding="utf-8")
    m = YOLO("yolo11m.pt")
    m.train(data=str(data_yaml), epochs=EPOCHS, imgsz=IMGSZ, batch=BATCH, device=DEVICE,
            project="runs", name=RUN_NAME, patience=30, seed=42, exist_ok=True,
            plots=True, verbose=True)
    WEIGHTS = str(Path(m.trainer.best))

print("Weights:", WEIGHTS)
print("Classes:", YOLO(WEIGHTS).names)

VIDEOS = {p.stem: p for p in VIDEO_DIR.glob("*.mp4")}
if not VIDEOS:
    up = files.upload()
    for fn in up:
        dst = VIDEO_DIR / Path(fn).name
        shutil.move(fn, dst)
        VIDEOS[dst.stem] = dst
print("Videos:", {k: str(v) for k, v in VIDEOS.items()})

TR_DIR = Path(ultralytics.__file__).parent / "cfg" / "trackers"
BASE = {t: yaml.safe_load((TR_DIR / f"{t}.yaml").read_text()) for t in ("bytetrack", "botsort")}

def make_cfg(name, tracker, overrides):
    cfg = dict(BASE[tracker])
    cfg.update(overrides)
    path = CFG_DIR / f"{name}.yaml"
    path.write_text(yaml.safe_dump(cfg, sort_keys=False), encoding="utf-8")
    return path

EXPERIMENTS = [
    ("byte_default",     "bytetrack", {}),
    ("byte_buffer10",    "bytetrack", {"track_buffer": 10}),
    ("byte_buffer90",    "bytetrack", {"track_buffer": 90}),
    ("byte_match0.6",    "bytetrack", {"match_thresh": 0.6}),
    ("byte_match0.95",   "bytetrack", {"match_thresh": 0.95}),
    ("byte_high0.5",     "bytetrack", {"track_high_thresh": 0.5, "new_track_thresh": 0.5}),
    ("byte_low0.3",      "bytetrack", {"track_low_thresh": 0.3}),
    ("byte_newtrack0.6", "bytetrack", {"new_track_thresh": 0.6}),
    ("bot_default",      "botsort",   {}),
    ("bot_gmc_none",     "botsort",   {"gmc_method": "none"}),
    ("bot_buffer10",     "botsort",   {"track_buffer": 10}),
    ("bot_buffer90",     "botsort",   {"track_buffer": 90}),
    ("bot_match0.6",     "botsort",   {"match_thresh": 0.6}),
    ("bot_match0.95",    "botsort",   {"match_thresh": 0.95}),
    ("bot_proximity0.3", "botsort",   {"proximity_thresh": 0.3}),
    ("bot_proximity0.7", "botsort",   {"proximity_thresh": 0.7}),
    ("bot_reid",         "botsort",   {"with_reid": True}),
]
CFG_PATHS = {n: make_cfg(n, t, o) for n, t, o in EXPERIMENTS}

def run_tracking(video_path, cfg_path):
    model = YOLO(WEIGHTS)
    track_len = collections.Counter()
    per_frame = []
    n_frames = 0
    t0 = time.time()
    gen = model.track(source=str(video_path), tracker=str(cfg_path), stream=True, persist=False,
                      conf=DET_CONF, iou=DET_IOU, imgsz=IMGSZ, device=DEVICE, verbose=False)
    for r in gen:
        n_frames += 1
        ids = []
        if r.boxes is not None and r.boxes.id is not None:
            ids = r.boxes.id.int().cpu().tolist()
            for tid in ids:
                track_len[tid] += 1
        per_frame.append(len(ids))
        if MAX_FRAMES and n_frames >= MAX_FRAMES:
```

```
            break
    dt = time.time() - t0
    lengths = np.array(list(track_len.values())) if track_len else np.array([0])
    max_simul = max(per_frame) if per_frame else 0
    return {
        "frames": n_frames,
        "unique_ids": len(track_len),
        "max_simultaneous": max_simul,
        "id_excess": len(track_len) - max_simul,
        "mean_objs_per_frame": round(float(np.mean(per_frame)) if per_frame else 0, 2),
        "mean_track_len": round(float(lengths.mean()), 1),
        "median_track_len": float(np.median(lengths)),
        "short_tracks": int((lengths < 15).sum()),
        "fps": round(n_frames / dt, 1),
    }

rows = []
for vname, vpath in VIDEOS.items():
    for exp_name, tracker, overrides in EXPERIMENTS:
        try:
            res = run_tracking(vpath, CFG_PATHS[exp_name])
        except Exception as e:
            print("skip", vname, exp_name, e)
            continue
        rows.append({"video": vname, "experiment": exp_name, "tracker": tracker,
                     "params": str(overrides) if overrides else "default", **res})
        print(vname, exp_name, res["unique_ids"], res["id_excess"], res["mean_track_len"], res["short_tracks"],
res["fps"])

df = pd.DataFrame(rows)
df.to_csv(OUT_DIR / "results.csv", index=False)
pd.set_option("display.width", 250)
pd.set_option("display.max_columns", 30)
print(df)

for vname in df["video"].unique():
    d = df[df["video"] == vname]
    fig, ax = plt.subplots(1, 3, figsize=(18, 5))
    cols = ["tab:blue" if t == "bytetrack" else "tab:orange" for t in d["tracker"]]
    for a, c, title in zip(ax, ["unique_ids", "id_excess", "mean_track_len"],
                           ["Unique IDs", "ID excess", "Mean track length"]):
        a.barh(d["experiment"], d[c], color=cols)
        a.set_title(f"{vname}: {title}")
        a.invert_yaxis()
        a.grid(alpha=0.3)
    plt.tight_layout()
    plt.savefig(OUT_DIR / f"metrics_{vname}.png", dpi=130)
    plt.show()

src = str(next(iter(VIDEOS.values())))
cap = cv2.VideoCapture(src)
fps = cap.get(5) or 25
W, H = int(cap.get(3)), int(cap.get(4))
cap.release()

def render(name, tracker=None, max_frames=MAX_FRAMES):
    model = YOLO(WEIGHTS)
    raw = str(OUT_DIR / f"{name}_raw.mp4")
    out = str(OUT_DIR / f"{name}.mp4")
    w = cv2.VideoWriter(raw, cv2.VideoWriter_fourcc(*"mp4v"), fps, (W, H))
    if tracker:
        gen = model.track(source=src, tracker=tracker, stream=True, persist=False,
                          conf=0.25, imgsz=IMGSZ, device=DEVICE, verbose=False)
    else:
        gen = model.predict(source=src, stream=True, conf=0.25, imgsz=IMGSZ, device=DEVICE, verbose=False)
    for i, r in enumerate(gen):
        w.write(r.plot())
        if i + 1 >= max_frames:
            break
    w.release()
    subprocess.run(["ffmpeg", "-y", "-loglevel", "error", "-i", raw,
                    "-vcodec", "libx264", "-pix_fmt", "yuv420p", out])
    os.remove(raw)
    return out

render("1_detection")
render("2_bytetrack", "bytetrack.yaml")
render("3_botsort", "botsort.yaml")
```

```
shutil.copytree(CFG_DIR, OUT_DIR / "tracker_cfgs", dirs_exist_ok=True)
shutil.make_archive("lab4_results", "zip", OUT_DIR)
files.download("lab4_results.zip")
```

Результат работы программы:

```
Using 3478 train, 389 val images for fraction=1.0 at imgsz=640
Using 2 dataloader workers
Logging results to /content/runs/detect/runs/yolo11m_watermeter
Starting training for 20 epochs...

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       1/20      7.88G      1.463      1.736      1.204         40        640: 100% ━━━━━━━━━━━━ 218/218 2.0it/s 1:49
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 1.9it/s 6.8s
                   all        389       2105      0.699      0.665      0.722       0.35

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       2/20      8.05G      1.399      1.014      1.144         51        640: 100% ━━━━━━━━━━━━ 218/218 2.0it/s 1:47
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.2it/s 5.8s
                   all        389       2105      0.734      0.728      0.779      0.366

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       3/20      8.08G      1.419     0.9311      1.159         42        640: 100% ━━━━━━━━━━━━ 218/218 2.0it/s 1:47
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.7s
                   all        389       2105      0.806      0.795      0.858      0.437

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       4/20      8.08G      1.378     0.8475       1.14         81        640: 100% ━━━━━━━━━━━━ 218/218 2.0it/s 1:47
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.7s
                   all        389       2105       0.89      0.857      0.905        0.5

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       5/20      8.08G      1.362     0.8042      1.133         51        640: 100% ━━━━━━━━━━━━ 218/218 2.0it/s 1:47
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.7s
                   all        389       2105      0.905      0.879      0.926      0.509

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       6/20      8.05G      1.341     0.7682      1.126         70        640: 100% ━━━━━━━━━━━━ 218/218 2.0it/s 1:47
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.7s
                   all        389       2105      0.904      0.847      0.921      0.493

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       7/20      8.07G      1.328     0.7205      1.114         57        640: 100% ━━━━━━━━━━━━ 218/218 2.0it/s 1:47
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.7s
                   all        389       2105      0.906      0.882      0.922      0.455

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       8/20      8.08G      1.295     0.6919      1.104         51        640: 100% ━━━━━━━━━━━━ 218/218 2.0it/s 1:47
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.7s
                   all        389       2105      0.917      0.908      0.939      0.511

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       9/20      8.08G      1.289      0.679      1.098         59        640: 100% ━━━━━━━━━━━━ 218/218 2.0it/s 1:48
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.8s
                   all        389       2105      0.909      0.905      0.937      0.522

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      10/20      8.05G      1.271     0.6527       1.09         42        640: 100% ━━━━━━━━━━━━ 218/218 2.1it/s 1:46
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.6s
                   all        389       2105      0.921       0.91      0.943      0.532
Closing dataloader mosaic
albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01,
method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8, 8))

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      11/20      8.05G      1.272     0.5863      1.144         32        640: 100% ━━━━━━━━━━━━ 218/218 2.0it/s 1:48
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.7s
                   all        389       2105      0.916      0.918      0.945      0.512

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      12/20      8.04G       1.25     0.5752      1.135         30        640: 100% ━━━━━━━━━━━━ 218/218 2.0it/s 1:47
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.7s
                   all        389       2105      0.927      0.924      0.942      0.507

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      13/20      8.06G      1.231     0.5534      1.129         30        640: 100% ━━━━━━━━━━━━ 218/218 2.1it/s 1:46
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.7s
                   all        389       2105      0.926      0.927      0.945       0.53

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      14/20      8.06G      1.212     0.5366      1.116         30        640: 100% ━━━━━━━━━━━━ 218/218 2.1it/s 1:43
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.5s
                   all        389       2105      0.932      0.928      0.951      0.545

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      15/20      8.06G      1.192     0.5209      1.101         28        640: 100% ━━━━━━━━━━━━ 218/218 2.2it/s 1:41
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.4it/s 5.5s
                   all        389       2105      0.926      0.942      0.959       0.55
```

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      16/20      8.06G      1.172     0.5067      1.094         30        640: 100% ━━━━━━━━━━━━ 218/218 2.2it/s 1:40
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.4it/s 5.4s
                   all        389       2105      0.928       0.93      0.958      0.553

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      17/20      8.06G      1.158     0.4945      1.087         28        640: 100% ━━━━━━━━━━━━ 218/218 2.1it/s 1:43
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.7s
                   all        389       2105      0.939       0.936     0.958      0.556

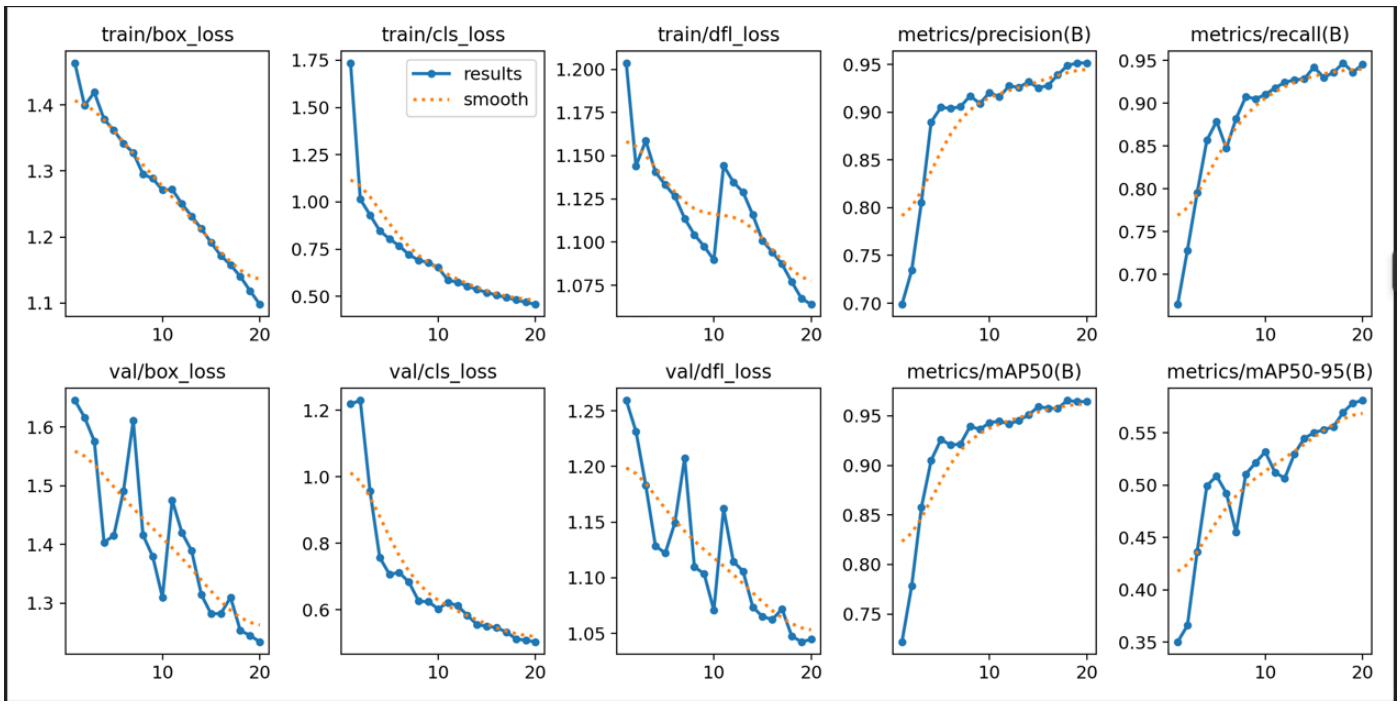
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      18/20      8.05G      1.14      0.4812      1.077         32        640: 100% ━━━━━━━━━━━━ 218/218 2.1it/s 1:46
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.7s
                   all        389       2105      0.949       0.947     0.966       0.57

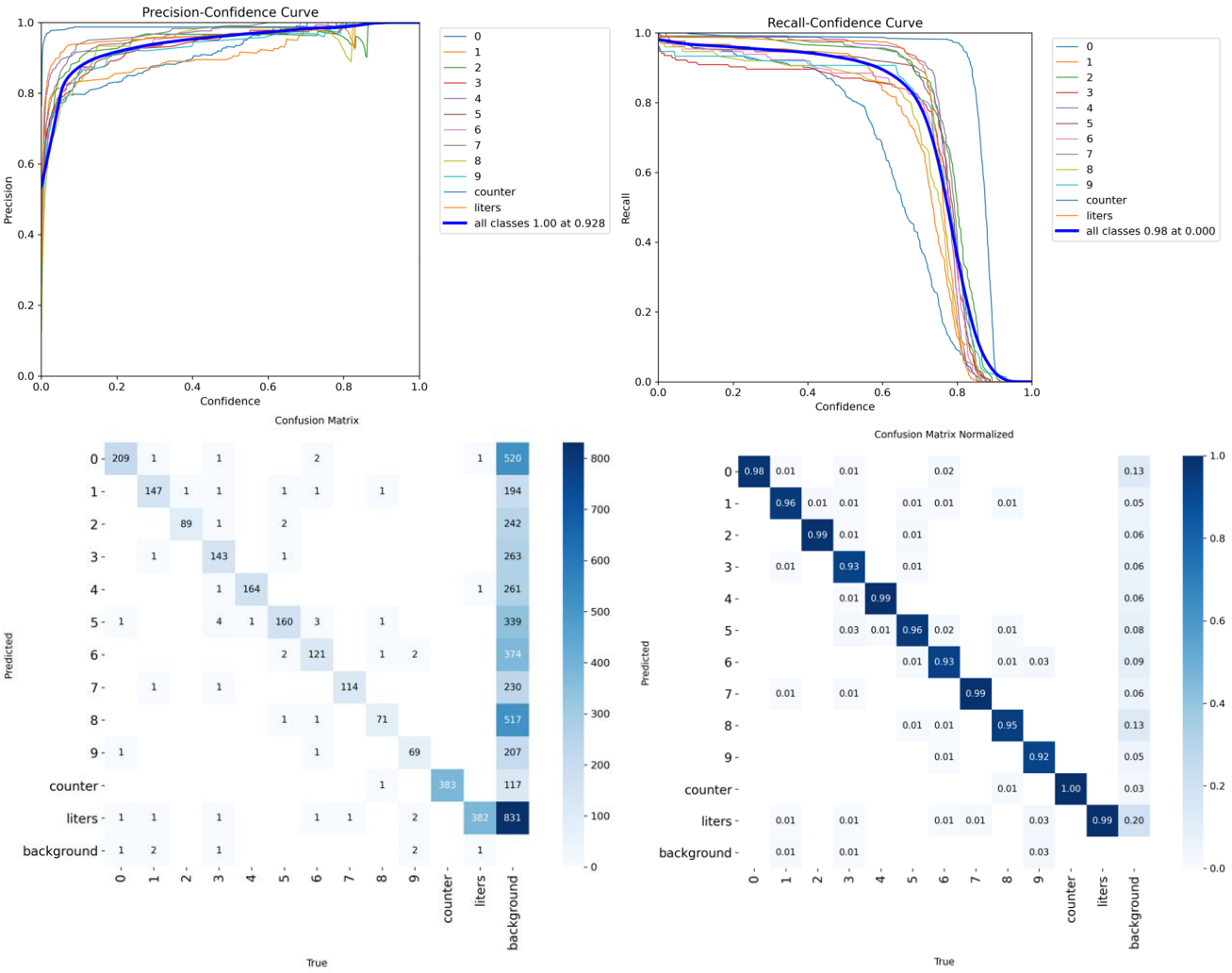
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      19/20      8.06G      1.119     0.469       1.067         29        640: 100% ━━━━━━━━━━━━ 218/218 2.0it/s 1:47
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.7s
                   all        389       2105      0.952       0.936     0.964      0.579

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      20/20      8.05G      1.098     0.4576      1.064         30        640: 100% ━━━━━━━━━━━━ 218/218 2.0it/s 1:47
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 2.3it/s 5.7s
                   all        389       2105      0.951       0.945     0.964      0.581

20 epochs completed in 0.661 hours.
Optimizer stripped from /content/runs/detect/runs/yolo11m_watermeter/weights/last.pt, 40.5MB
Optimizer stripped from /content/runs/detect/runs/yolo11m_watermeter/weights/best.pt, 40.5MB

Validating /content/runs/detect/runs/yolo11m_watermeter/weights/best.pt...
Ultralytics 8.4.197 🚀 Python-3.13.16 torch-2.11.0+cu130 CUDA:0 (Tesla T4, 14913MiB)
YOLO11m summary (fused): 125 layers, 20,039,284 parameters, 0 gradients, 67.8 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 13/13 1.5it/s 8.5s
                   all        389       2105      0.952       0.945     0.964      0.581
                     0        162        213      0.895       0.901     0.946      0.528
                     1        108        153       0.89       0.948      0.95      0.535
                     2         87         90      0.967       0.966     0.968      0.539
                     3        125        154      0.944       0.896      0.94      0.567
                     4        132        165      0.972       0.988     0.987       0.63
                     5        119        167      0.955       0.946      0.97      0.551
                     6        112        130       0.96        0.92     0.968      0.581
                     7        100        115      0.982       0.976     0.994       0.57
                     8         64         75      0.977       0.907     0.952      0.539
                     9         70         75      0.932        0.92     0.932       0.58
               counter        371        383      0.987       0.989     0.992      0.795
                liters        377        385      0.961       0.979     0.974      0.563
Speed: 1.2ms preprocess, 13.5ms inference, 0.0ms loss, 1.5ms postprocess per image
```




Precision-Confidence Curve
Precision
Confidence
0
1
2
3
4
5
6
7
8
9
counter
liters
all classes 1.00 at 0.928
Recall-Confidence Curve
Recall
Confidence
all classes 0.98 at 0.000
Confusion Matrix
Predicted
True
background
Confusion Matrix Normalized

Эталон:


counter
liters

Предсказания модели:



Результат видеодетекции:



Эксперимент	Уникальные ID	Избыток ID	Среднее объектов/кадр	Средняя длина трека, кадров	Короткие треки < 15 кадров	FPS
byte_default	8	5	2,85	213,8	2	38,2
byte_buffer10	12	9	2,83	141,5	4	38,4
byte_buffer90	5	2	2,87	344,4	1	38,0
byte_match0.6	11	8	2,82	153,8	3	38,1
byte_match0.95	6	3	2,86	286,0	1	38,3
byte_high0.5	7	4	2,60	222,9	1	38,5
byte_low0.3	10	7	2,72	163,2	3	38,3
byte_newtrack0.6	6	3	2,69	269,0	1	38,4
bot_default	6	3	2,88	288,0	1	35,1
bot_gmc_none	9	6	2,81	187,3	2	36,7
bot_buffer10	10	7	2,84	170,4	3	35,2
bot_buffer90	4	1	2,90	435,0	0	35,0
bot_match0.6	9	6	2,83	188,7	2	35,2
bot_match0.95	5	2	2,89	346,8	1	35,1
bot_proximity0.3	6	3	2,88	288,0	1	35,0
bot_proximity0.7	6	3	2,88	288,0	1	35,2
bot_reid	4	1	2,89	433,5	0	27,6

При настройках по умолчанию ByteTrack выдал 8 ID (избыток 5, средняя длина трека 213,8 кадра, 38,2 FPS), BoT-SORT: 6 ID (избыток 3, длина 288,0 кадра, 35,1 FPS). BoT-SORT стабильнее, ByteTrack быстрее примерно на 3 FPS.
Влияние параметров:

- **track_buffer.** Основной параметр. Увеличение до 90 снизило число ID у ByteTrack с 8 до 5, у BoT-SORT с 6 до 4, а длину трека увеличило до 344,4 и 435,0 кадра. Уменьшение до 10 вызвало фрагментацию: 12 ID у ByteTrack и 10 у BoT-SORT.
- **match_thresh.** Снижение до 0,6 ухудшило результат (11 и 9 ID), повышение до 0,95 улучшило (6 и 5 ID), так как более строгое сопоставление реже склеивает разные объекты.
- **track_low_thresh и new_track_thresh (ByteTrack).** При track_low_thresh = 0,3 получено 10 ID (хуже), при new_track_thresh = 0,6 6 ID (лучше). Повышение track_high_thresh до 0,5 дало 7 ID и снизило среднее число объектов в кадре до 2,60, то есть часть объектов терялась.

- **gmc_method (BoT-SORT).** Отключение компенсации движения камеры ухудшило результат с 6 до 9 ID, значит, камера в видео движется.
- **proximity_thresh.** Результат не изменился (6 ID при 0,3, 0,5 и 0,7): параметр действует только при включённом ReID.
- **ReID.** Дал 4 ID (избыток 1) при длине трека 433,5 кадра, но скорость упала с 35,1 до 27,6 FPS.

Лучший результат по качеству: BoT-SORT с track_buffer = 90 (4 ID, избыток 1, длина трека 435,0, ни одного короткого трека, 35,0 FPS). BoT-SORT с ReID даёт такое же качество, но дороже по скорости. Для ByteTrack лучшей оказалась track_buffer = 90 (5 ID). Главные параметры: track_buffer и match_thresh; для движущейся камеры важна компенсация движения.

Вывод: исследовал применение алгоритмов трекинга на базе обученной сети-детектора объектов.